

On Empirical Memory Design, Faster Selection of Bayesian Factorizations and Parameter-Free Gaussian EDAs

Peter A.N. Bosman
Centre for Mathematics and Computer Science
P.O. Box 94079
1090 GB Amsterdam
The Netherlands
Peter.Bosman@cwi.nl

ABSTRACT

Often, Estimation-of-Distribution Algorithms (EDAs) are praised for their ability to optimize a broad class of problems. EDA applications are however still limited. Often heard criticism is that a large population size is required and that distribution estimation takes long. Here we look at possibilities for improvements in these areas. We first discuss the use of a memory to aggregate information over multiple generations and reduce the population size. The approach we take, empirical risk minimization to perform non-linear regression of memory parameters, may well be generalizable to other EDAs. We design a memory this way for a Gaussian EDA and observe smaller population size requirements and fewer evaluations used. We also speed up the selection of Bayesian factorizations for Gaussian EDAs by sorting the entries in the covariance matrix. Finally, we discuss parameter-free Gaussian EDAs for real-valued single-objective optimization. We propose to not only increase the population size in subsequent runs, but to also divide it over parallel runs across the search space. On some multimodal problems improvements are thereby obtained.

Categories and Subject Descriptors

G.1 [Numerical Analysis]: Optimization; I.2 [Artificial Intelligence]: Problem Solving, Control Methods, and Search

General Terms

Algorithms, Optimization, Performance, Experimentation

Keywords

Evolutionary Algorithms, Estimation of Distribution Algorithms, Numerical Optimization, Memory, Learning

1. INTRODUCTION

Estimation-of-Distribution Algorithms (EDAs) are an important area of fundamental research and a source of innovation in the field of Genetic and Evolutionary Algorithms

(GEAs). The statistical use of probability distributions is an elegant and powerful way to detect and exploit dependencies between problem variables [11, 13]. EDAs can thereby avoid encoding deceptions; classic GEAs typically can't.

Applications of research are equally important. EDA applications are however still limited. Characteristics of real-world problems that are often mentioned in the literature are dynamic changes [5] and multiple objectives [7]. Efficient and effective GEAs designed for such problems typically have small population sizes and multiple subpopulations, e.g. to search in parallel for dynamic changes in various parts of the search space [6] or to spread the search along a Pareto front [4]. Ease of use also contributes to applicability. EDAs and GEAs typically have many parameters. Removing them greatly eases the use of EDAs.

Our aim is to make EDAs more fit for applications: a smaller population size, faster model building and easy use. First, in Section 2 we discuss the use of a memory. Selected solutions of subsequent generations typically have a lot in common. Re-estimating from scratch in each generation, as is usually done in EDAs, is therefore likely not the most efficient approach. A memory can be used to store and update information of previous generations. This reduces the population size required to estimate the distribution properly. Memories have been used in prominent EDAs such as (i)BOA [14] and CMA-ES [9] before. In both cases, the required population size is reduced. The design of the memory in these cases is however EDA-specific. Here, we take an approach that may well be generalized to other EDAs.

Often heard criticism on EDAs is that distribution estimation takes long. For instance, the learning of factorizations takes much longer than the application of classical crossover. Sometimes (e.g. for real-world problems), this time may be subsumed by the time required to evaluate a solution. Then still, if the computational complexity of distribution estimation is large, the required time is likely to outgrow the time required to evaluate a solution as the problem dimensionality l increases. The importance of this issue has been acknowledged before. Recently for instance, the computational complexity of factorization selection in ECGA was brought back from $\mathcal{O}(l^3)$ to $\mathcal{O}(l^2)$ [8]. In Section 3, we present a reduction in the computational complexity of Bayesian factorization selection for the normal distribution.

Finally, in Section 4 we discuss the ease-of-use issue. Several parameter-free GEAs and EDAs exist (see, e.g. [1, 10, 12]). The population size is one of the most important parameters. Often, a scheme is employed in which the popu-

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

GECCO'09, July 8–12, 2009, Montréal, Québec, Canada.

Copyright 2009 ACM 978-1-60558-325-9/09/07 ...\$5.00.

lation size increases exponentially. With a small population the algorithm is expected to terminate quickly, wasting only few evaluations if the optimum is not found. For real-valued optimization with the normal distribution however, enlarging the population size may not always be the most effective approach because the single normal distribution essentially represents a centered approach. We therefore look at a different way to make a Gaussian EDA parameter-free.

2. EMPIRICAL MEMORY DESIGN

2.1 General Design

Let θ be a subset of all distribution parameters. For the normal distribution θ could be e.g. the covariance matrix. For a (factorized) probability distribution over discrete variables, θ could be all possible parameters. Note that this doesn't mean that all distribution parameters actually have to be computed. A typical formula for using a memory in a learning task based on memory decay is the following:

$$\theta(t) = (1 - \eta)\theta(t-1) + \eta\hat{\theta}(t) \quad (1)$$

where $\hat{\theta}(t)$ is the result of estimating parameters θ from the data in generation t and $\eta \in [0, 1]$ is called the learning rate. A similar formula is used in some of the first EDAs, i.e. in PBIL [2] and its continuous counterpart PBIL_C [18]. Obtaining $\hat{\theta}(t)$ from the set of selected solutions can be done in the same way as is done without a memory. For instance, $\hat{\theta}(t)$ can be a maximum-likelihood fit.

For each subset of parameters θ , a different learning rate η is to be determined. Because it is not always straightforward to do so analytically, we want to see whether an empirical approach, that is, learning from data, is possible. We must therefore first determine the function class. $\theta(t)$ plays a role in the determination of $\theta(t')$, $t' > t$. After $t^{F_{ade}}$ generations, the contribution of $\theta(t)$ has faded to a contribution level of $z = (1 - \eta)^{t^{F_{ade}}}$, giving $t^{F_{ade}} = \log(z)/\log(1 - \eta)$. Furthermore, the population size and therefore also selection size requirements for an EDA without a memory is typically bounded by a polynomial in l [13]. The selection size required for determining distribution parameters can therefore be bounded by cl^d . The generational selection size is $|\mathcal{S}(t)|$ where $\mathcal{S}(t)$ is a vector of selected solutions in generation t . Selected individuals in generation t contribute with different weight to multiple generations $t' > t$. The overall selection size to estimate distribution parameters is therefore likely bounded by $a|\mathcal{S}|^b$, where we assume the selection size in each generation to be the same. For the overall selection size then to equal the required selection size, $t^{Sel} = (c/a)(l^d/|\mathcal{S}|^b)$ generations are required. In the next $t^{F_{ade}}$ generations, the overall selection size must be large enough to derive new distribution parameters. This means equating t^{Sel} and $t^{F_{ade}}$, which gives $\log(1 - \eta) = \log(z)(a/c)(|\mathcal{S}|^b/l^d)$. We thereby obtain a function class with three parameters α_0, α_1 and α_2 and two variables $|\mathcal{S}|$ and l that is likely to allow for a good fit of η to the data. Let $\alpha = (\alpha_0, \alpha_1, \alpha_2)$. Then we have:

$$\eta = f_{\alpha}^{\eta}(|\mathcal{S}|, l) = 1 - \exp(\alpha_0|\mathcal{S}|^{\alpha_1}/l^{\alpha_2}) \quad (2)$$

Next, a careful selection of representative problems is required. On these problems, for various values of the variables of the function class (i.e. $|\mathcal{S}|$ and l), the best value for η must be found. This creates a dataset of n^{Pat} patterns $\{(\eta^i, (|\mathcal{S}|^i, l^i))\}$, $i \in \{0, 1, \dots, n^{\text{Pat}} - 1\}$. Through regression,

values for α can be found using empirical risk minimization with the squared-loss function [19]. This amounts to solving the following optimization problem:

$$\min_{\alpha} \sum_{i=0}^{n^{\text{Pat}}-1} \left(\eta^i - f_{\alpha}^{\eta}(|\mathcal{S}|^i, l^i) \right)^2 \quad (3)$$

This problem can typically not be solved analytically (e.g. due to the non-linearity of the function class). A non-linear optimization technique is therefore required.

2.2 Application to a Gaussian EDA

2.2.1 Base EDA

Currently, the CMA-ES [9] is the most efficient EDA based on the normal or Gaussian distribution. Although it has a historic relation with Evolution Strategies (ES), it is very similar to other Gaussian EDAs. CMA-ES has a strong theoretic background with analytical derivations for most of its formulae. It also already makes use of a memory. For this reason we focus on another Gaussian EDA: AMaL-GaM [3]. This EDA uses maximum-likelihood estimates for the mean and the covariance matrix, estimated from the selected solutions. It has a mechanism that scales up the covariance matrix when required to prevent premature convergence on slopes. It furthermore has a mechanism that anticipates the mean shift in the next generation to speed up descent (in case of minimization) along slopes. For more details we refer the interested reader to the literature [3].

Variants of the same EDA can be obtained using factorizations. Factorizations are used to reduce the effort of learning the full multivariate probability distribution. Initial EDAs that used the normal distribution employed the univariate factorization [16, 18]. In the univariate factorization, the distribution is estimated and sampled independently for each variable. We will call such EDAs *naive* EDAs. Let $\mathcal{X} = (X_0, X_1, \dots, X_{l-1})$ be a vector containing all random variables. The univariate factorization then is defined as $P(\mathcal{X}) = \prod_{i=0}^{l-1} P(X_i)$.

Bayesian factorizations are often used to incorporate a subset of all possible dependencies. To briefly recall Bayesian factorizations, recall that the vector of random variables X_{π_i} on which X_i is conditioned in the Bayesian factorization is called the vector of parents of X_i . A Bayesian factorization can now be written as $P(\mathcal{X}) = \prod_{i=0}^{l-1} P(X_i|X_{\pi_i})$. A greedy algorithm can be used to select the parent configurations in the Bayesian factorization. This is a quite common approach in EDAs (for more details, see e.g. [11, 13]).

Because the number of parameters of the full, unfactorized normal distribution is quadratic, it is not intractable to use it in an EDA. The advantage is that it incorporates all possible dependencies. The density contours of a normal distribution are ellipsoids. Therefore, only linear dependencies can be modelled efficiently. If all possible dependencies are allowed (i.e. the full covariance matrix), these ellipsoids can be aligned along any axis. If there is no dependency between a set of random variables (i.e. the univariate factorization), the density contours in those dimensions can only be aligned with the main axes.

2.2.2 Distribution parameters for the memory

Most of the parameters are stored in the covariance matrix Σ . The covariance matrix is also the main reason why the population size has to grow faster with l than when only

the variances are considered. More samples are required for the maximum-likelihood estimate of the covariance matrix to ensure that it is conditioned well.

With a memory for the covariance matrix the population size requirement is expected to be smaller. For AMaLGaM, it is then also important to have a memory for the anticipated mean shift (AMS). In AMaLGaM, the AMS is computed as the difference between the means of subsequent generations, i.e. $\hat{\mu}^{\text{Shift}}(t) = \hat{\mu}(t) - \hat{\mu}(t-1)$. A part of the newly sampled individuals is then moved in the direction of the AMS: $\mathbf{x} \leftarrow \mathbf{x} + 2\hat{\mu}^{\text{Shift}}(t)$. If the population size is small, the estimation of the mean becomes less reliable. Having a memory for the mean is however undesirable. If the mean needs to shift much before centering over a (local) optimum, a memory will slow this shift down. A memory for the AMS is desirable however. With less reliable mean estimates, the AMS may oscillate much around its optimal direction. With a memory these oscillations can be smoothed out.

This gives two sets of distribution parameters to build a memory for. Summarizing, the estimations are now:

$$\begin{aligned}\hat{\mu}(t) &= \frac{1}{|\mathcal{S}|} \sum_{i=0}^{|\mathcal{S}|-1} \mathcal{S}_i \\ \hat{\Sigma}(t) &= (1-\eta^{\Sigma})\hat{\Sigma}(t-1) + \\ &\quad \eta^{\Sigma} \frac{1}{|\mathcal{S}|} \sum_{i=0}^{|\mathcal{S}|-1} (\mathcal{S}_i - \hat{\mu}(t)) (\mathcal{S}_i - \hat{\mu}(t))^T \quad (4) \\ \hat{\mu}^{\text{Shift}}(t) &= (1-\eta^{\text{Shift}})\hat{\mu}^{\text{Shift}}(t-1) + \\ &\quad \eta^{\text{Shift}} (\hat{\mu}(t) - \hat{\mu}(t-1))\end{aligned}$$

Moreover, for $\hat{\Sigma}(0)$ we use a matrix with the maximum-likelihood variances on the diagonal and zeros off the diagonal. Also, $\hat{\mu}^{\text{Shift}}(t)$ is non-existent for $t = 0$ and for $t = 1$ it is just the equation without memory, i.e. $\hat{\mu}(1) - \hat{\mu}(0)$.

We finally note that the metric in the greedy algorithm for selecting the dependencies of the Bayesian factorization was changed. Originally, the metric used is the *Bayesian Information Criterion* (BIC) [17]. This metric is quite commonly used in EDAs. It scores a probability distribution by its negative log-likelihood, but adds a penalty term that increases with the model complexity $|\theta|$ and the size of the sample vector $|\mathcal{S}|$: $-\sum_{i=0}^{|\mathcal{S}|-1} \ln(P(\mathbf{x}^i)) + \frac{1}{2} \ln(|\mathcal{S}|) |\theta|$. Here, θ is the number of distribution parameters to be estimated. Because a memory is used for the covariance matrix, the distribution describes not only the current set of selected solutions but to a certain extent also previous sets of selected solutions. This adds noise to the error term because it is only computed for the current selection. For this reason, we reduce the complexity term that represents the threshold for adding a parent in the Bayesian factorization by multiplying it by η^{Σ} , i.e. we use a complexity term of $\frac{1}{2} \eta^{\Sigma} \ln(|\mathcal{S}|) |\theta|$.

2.2.3 Learning the memory parameters

To learn the memory parameters we used the following set of ten common benchmark problems to be minimized:

Name	Definition	VTR
Sphere	$\sum_{i=0}^{l-1} x_i^2$	10^{-10}
Ellipsoid	$\sum_{i=0}^{l-1} 10^{6 \frac{i}{l-1}} x_i^2$	10^{-10}
Cigar	$x_0^2 + \sum_{i=1}^{l-1} 10^6 x_i^2$	10^{-10}
Tablet	$10^6 x_1^2 + \sum_{i=1}^{l-1} x_i^2$	10^{-10}
Cigar Tablet	$x_0^2 + \sum_{i=1}^{l-2} 10^4 x_i^2 + 10^8 x_{l-1}^2$	10^{-10}
Two Axes	$\sum_{i=0}^{\lfloor l/2 \rfloor - 1} 10^6 x_i^2 + \sum_{i=\lfloor l/2 \rfloor - 1}^{l-1} x_i^2$	10^{-10}
Different Powers	$\sum_{i=0}^{l-1} x_i ^{2+10 \frac{i}{l-1}}$	10^{-15}

	Naive		Bayesian		Full	
	η^{Σ}	η^{Shift}	η^{Σ}	η^{Shift}	η^{Σ}	η^{Shift}
α_0	-0.40	-0.31	-0.33	-0.52	-1.1	-1.2
α_1	0.15	0.27	1.5	0.70	1.2	0.31
α_2	-0.034	0.067	1.1	0.65	1.6	0.50

Table 1: Regressed memory parameters for each variant of AMaLGaM.

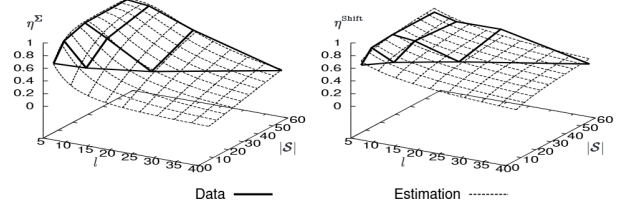


Figure 1: Regression results for the memory function for the covariance matrix (left) and the AMS (right) for the full version of AMaLGaM.

Name	Definition	VTR
Rosenbrock	$\sum_{i=0}^{l-2} (100 \cdot (x_i^2 - x_{i+1})^2 + (x_i - 1)^2)$	10^{-10}
Parabolic Ridge	$-x_1 + 100 \sum_{i=1}^{l-1} x_i^2$	-10^{10}
Sharp Ridge	$-x_1 + 100 \sqrt{\sum_{i=1}^{l-1} x_i^2}$	-10^{10}

For each problem, dimensionalities $l \in \{5, 10, 20, 40\}$ and population sizes $n \in \{10, 20, 40, 80, 160\}$ were used. For each combination of l and n the best value for η^{Σ} and η^{Shift} was determined as the combination that leads to the minimum average number of evaluations to reach the value-to-reach (VTR), averaged over 10 independent runs. To perform this two-dimensional real-valued optimization task, we used the original AMaLGaM. Using the so-obtained datasets, we again used the original AMaLGaM, but this time to solve Equation 3. For both uses, the original AMaLGaM was found to give robust and repeatable results. For each variant of AMaLGaM, the resulting values for the three parameters of the function class for the two different functions are presented in Table 1. An illustration of the regressed functions for η^{Σ} and η^{Shift} for the case of using the full covariance matrix are presented in Figure 1.

2.2.4 Before memory, after memory

The set of possible dimensionalities for testing in this Section is $l \in \{2, 4, 8, 10, 20, 40, 80, 100, 200, 400, 800, 1000\}$. First, for all problems in Section 2.2.3 the population size was determined that resulted in the minimum average number of evaluations, averaged over 10 independent runs. Because time consumption increases with EDA model complexity, a different maximum for l was used for each variant of AMaLGaM: naive: 400, Bayesian: 200, full: 100. Furthermore, for the case of the Bayesian factorization, we set the maximum number of parents for any variable to $\kappa = 5$. We ran tests for different initialization ranges (IRs). Asymmetrical around the optimum, $[-10; 5]$, and a far away from the optimum, $[-115; -100]$. Results for the asymmetric IR are subsumed by results for the far-away IR. For this reason and because use of the far-away IR prevents a possible contraction-bias to influence the results, we only report results for the far-away IR.

Figure 2 shows the population sizes that were found. Also shown are results for AMaLGaM without memory. We re-

computed those results because some details were changed compared to its original published version in the light of the final parameter-free version. More details on the changes are given in Section 4.1. From these results, we extracted new guidelines for the minimally required population size. For the versions that use the memory, the guideline now always scales with the square root of l . For the full covariance matrix this means a substantial complexity improvement. In the remainder, we will refer to the version of AMaLGaM with a memory as incremental AMaLGaM (iAMaLGaM).

- Full covariance matrix (unfactorized)
AMaLGaM: $n \geq 17 + 3l^{1.5}$, iAMaLGaM: $n \geq 10l^{0.5}$
- Factorization selection (Bayesian factorization)
AMaLGaM: $n \geq 12 + 8l^{0.7}$, iAMaLGaM: $n \geq 7l^{0.5}$
- Naive (univariate factorization)
AMaLGaM: $n \geq 10l^{0.5}$, iAMaLGaM: $n \geq 4l^{0.5}$

With these guidelines (i)AMaLGaM new results were computed over 100 independent runs. Because now only a single population size needs to be used for each value of l , much larger dimensionalities can be tested: naive: 1000, Bayesian: 400, full: 200. Figure 3 shows the ratio of the number of evaluations required by iAMaLGaM compared to AMaLGaM. For the naive case the ratio is roughly 50% for most problems. For the full covariance matrix the ratio is even more favorable. For the Bayesian factorization, the improvement in the range up to $l = 400$ is smaller, but still present. With increasing dimensionality the results of iAMaLGaM come closer to those of AMaLGaM. These less favorable results are likely the result of too little data used in the regression. Overall, the results indicate that a memory can be effectively configured using the suggested empirical approach.

For completeness, Figure 4 shows graphs for the number of evaluations required by iAMaLGaM. Furthermore, Table 2 provides scalability coefficients obtained from regressing αl^β on a log-log scale for the number of evaluations and the actual running time on a core that is average by today's standards (AMD Athlon 64 3500+, 2.2 Ghz). The code used is not especially optimized. The coefficients are computed from results in a much larger dimensionality-range than earlier results [3], making them more precise. Also shown is the extrapolated maximum dimensionality for which the problem is expected to be solved within 24 hours. It should be noted that the CMA-ES implementation used is in MATLAB. The computational overhead is therefore different for this method. Time-wise, only the exponent column (β^{Time}) is therefore comparable to the other results. Although CMA-ES remains the overall best, especially in terms of evaluations-complexity, AMaLGaM and iAMaL-GaM are comparable. The naive variant is able tackle problems of much larger l within 24 hours. Also, assuming a linear time-increase in evaluations, iAMaLGaM is able to solve a problem of roughly twice the dimensionality that AMaLGaM can solve in 24 hours. This depends however on the time required to perform an evaluation, which, for a real-world problem, is likely to be far more time-consuming.

On some problems, the results for the naive iAMaLGaM are similar to those of the analytical sep-CMA-ES [15] (a univariately factorized variant of CMA-ES). This supports the conclusion that using the suggested approach to empirical memory design in EDAs, very good results are possible.

Function	EDA	α^{Evals}	$\beta^{\text{Ev.}}$	α^{Time}	β^{Time}	l_{Day}
Sphere	AM-N	$2.53 \cdot 10^2$	1.18	$9.64 \cdot 10^{-5}$	2.16	13986
	iAM-N	$2.15 \cdot 10^2$	1.11	$9.38 \cdot 10^{-5}$	2.07	21582
	AM-B	$3.17 \cdot 10^2$	1.27	$4.11 \cdot 10^{-5}$	2.89	1670
	iAM-B	$1.78 \cdot 10^2$	1.36	$3.28 \cdot 10^{-5}$	3.01	1360
	AM-F	$1.50 \cdot 10^2$	1.91	$1.42 \cdot 10^{-5}$	3.75	404
	iAM-F	$9.36 \cdot 10^1$	1.81	$5.57 \cdot 10^{-5}$	3.15	825
	CMA	$3.21 \cdot 10^2$	0.87	$3.51 \cdot 10^{-6}$	3.15	1990
Ellipsoid	AM-N	$3.31 \cdot 10^2$	1.18	$2.02 \cdot 10^{-4}$	2.14	10714
	iAM-N	$2.58 \cdot 10^2$	1.11	$1.52 \cdot 10^{-4}$	2.09	15376
	AM-B	$4.35 \cdot 10^2$	1.25	$6.77 \cdot 10^{-5}$	2.85	1558
	iAM-B	$2.24 \cdot 10^2$	1.35	$1.60 \cdot 10^{-4}$	2.73	1579
	AM-F	$1.75 \cdot 10^2$	1.94	$3.55 \cdot 10^{-5}$	3.61	397
	iAM-F	$1.21 \cdot 10^2$	1.86	$1.66 \cdot 10^{-4}$	3.02	765
	CMA	$9.70 \cdot 10^1$	1.83	$9.07 \cdot 10^{-6}$	4.12	264
Cigar	AM-N	$3.44 \cdot 10^2$	1.18	$1.46 \cdot 10^{-4}$	2.16	11506
	iAM-N	$2.95 \cdot 10^2$	1.10	$1.46 \cdot 10^{-4}$	2.05	19233
	AM-B	$4.26 \cdot 10^2$	1.27	$5.35 \cdot 10^{-5}$	2.90	1500
	iAM-B	$2.39 \cdot 10^2$	1.36	$1.42 \cdot 10^{-4}$	2.77	1495
	AM-F	$2.06 \cdot 10^2$	1.91	$2.56 \cdot 10^{-5}$	3.68	387
	iAM-F	$1.28 \cdot 10^2$	1.85	$1.77 \cdot 10^{-4}$	3.00	795
	CMA	$6.75 \cdot 10^2$	0.91	$2.78 \cdot 10^{-4}$	2.54	2232
Tablet	AM-N	$2.79 \cdot 10^2$	1.17	$8.71 \cdot 10^{-5}$	2.19	12708
	iAM-N	$2.23 \cdot 10^2$	1.10	$9.51 \cdot 10^{-5}$	2.07	21312
	AM-B	$4.10 \cdot 10^2$	1.22	$4.63 \cdot 10^{-5}$	2.87	1697
	iAM-B	$1.95 \cdot 10^2$	1.34	$3.39 \cdot 10^{-5}$	3.01	1348
	AM-F	$1.59 \cdot 10^2$	1.90	$2.31 \cdot 10^{-5}$	3.64	424
	iAM-F	$1.32 \cdot 10^2$	1.75	$1.42 \cdot 10^{-4}$	2.95	947
	CMA	$1.86 \cdot 10^2$	1.54	$7.46 \cdot 10^{-6}$	3.88	389
Cigar - Tablet	AM-N	$3.53 \cdot 10^2$	1.17	$1.15 \cdot 10^{-4}$	2.19	11220
	iAM-N	$2.72 \cdot 10^2$	1.10	$1.06 \cdot 10^{-4}$	2.08	18865
	AM-B	$4.64 \cdot 10^2$	1.24	$5.53 \cdot 10^{-5}$	2.88	1555
	iAM-B	$2.31 \cdot 10^2$	1.35	$1.33 \cdot 10^{-4}$	2.76	1577
	AM-F	$2.06 \cdot 10^2$	1.90	$2.88 \cdot 10^{-5}$	3.65	397
	iAM-F	$1.44 \cdot 10^2$	1.80	$1.79 \cdot 10^{-4}$	2.96	858
	CMA	$3.30 \cdot 10^2$	1.31	$6.42 \cdot 10^{-5}$	3.21	699
Two - Axes	AM-N	$3.40 \cdot 10^2$	1.18	$1.04 \cdot 10^{-4}$	2.21	11059
	iAM-N	$2.77 \cdot 10^2$	1.10	$1.10 \cdot 10^{-4}$	2.08	18703
	AM-B	$4.45 \cdot 10^2$	1.25	$5.73 \cdot 10^{-5}$	2.88	1540
	iAM-B	$2.32 \cdot 10^2$	1.35	$4.56 \cdot 10^{-5}$	2.99	1257
	AM-F	$1.68 \cdot 10^2$	1.95	$2.37 \cdot 10^{-5}$	3.69	389
	iAM-F	$1.10 \cdot 10^2$	1.92	$8.63 \cdot 10^{-5}$	3.19	662
	CMA	$2.43 \cdot 10^2$	1.66	$1.03 \cdot 10^{-4}$	3.62	293
Different - Powers	AM-N	$1.93 \cdot 10^2$	1.29	$1.17 \cdot 10^{-4}$	2.28	7668
	iAM-N	$1.48 \cdot 10^2$	1.27	$1.01 \cdot 10^{-4}$	2.23	10131
	AM-B	$1.74 \cdot 10^2$	1.43	$2.60 \cdot 10^{-5}$	3.05	1333
	iAM-B	$1.52 \cdot 10^2$	1.35	$1.74 \cdot 10^{-4}$	2.66	1850
	AM-F	$8.13 \cdot 10^1$	2.09	$1.13 \cdot 10^{-5}$	3.85	366
	iAM-F	$8.82 \cdot 10^1$	1.92	$1.65 \cdot 10^{-4}$	3.02	771
	CMA	$2.34 \cdot 10^2$	1.59	$7.57 \cdot 10^{-5}$	3.59	331
Rosenbrock	AM-B	$3.38 \cdot 10^2$	1.70	$6.37 \cdot 10^{-5}$	3.26	639
	iAM-B	$1.34 \cdot 10^2$	1.76	$5.50 \cdot 10^{-5}$	3.29	629
	AM-F	$8.98 \cdot 10^1$	2.42	$1.20 \cdot 10^{-5}$	4.18	228
	iAM-F	$1.18 \cdot 10^2$	2.08	$2.30 \cdot 10^{-4}$	3.21	467
	CMA	$1.26 \cdot 10^2$	1.82	$9.31 \cdot 10^{-6}$	4.19	238
Parabolic - Ridge	AM-N	$1.65 \cdot 10^2$	1.01	$6.45 \cdot 10^{-5}$	1.99	38934
	iAM-N	$1.43 \cdot 10^2$	0.98	$1.01 \cdot 10^{-4}$	1.88	57803
	AM-B	$2.79 \cdot 10^2$	1.09	$2.95 \cdot 10^{-5}$	2.75	2753
	iAM-B	$1.88 \cdot 10^2$	1.17	$7.44 \cdot 10^{-5}$	2.71	2218
	AM-F	$1.18 \cdot 10^2$	1.91	$1.43 \cdot 10^{-5}$	3.68	452
	iAM-F	$4.57 \cdot 10^1$	1.83	$1.73 \cdot 10^{-4}$	2.76	1413
	CMA	$5.29 \cdot 10^2$	0.96	$1.94 \cdot 10^{-4}$	2.61	2066
Sharp - Ridge	AM-N	$1.37 \cdot 10^2$	0.93	$2.28 \cdot 10^{-4}$	1.69	118028
	iAM-N	$8.20 \cdot 10^1$	0.99	$1.09 \cdot 10^{-4}$	1.78	101840
	AM-B	$1.86 \cdot 10^2$	1.03	$3.80 \cdot 10^{-5}$	2.58	4168
	iAM-B	$8.02 \cdot 10^1$	1.23	$5.27 \cdot 10^{-5}$	2.67	2799
	AM-F	$8.07 \cdot 10^1$	1.83	$8.85 \cdot 10^{-6}$	3.63	569
	iAM-F	$6.82 \cdot 10^1$	1.72	$9.91 \cdot 10^{-5}$	2.86	1326
	CMA	$1.19 \cdot 10^2$	1.67	$9.39 \cdot 10^{-5}$	3.28	535

Table 2: Regressed scalability coefficients and maximum expected solvable dimensionality per problem and per EDA. IR is $[-115; -100]$ for each variable.

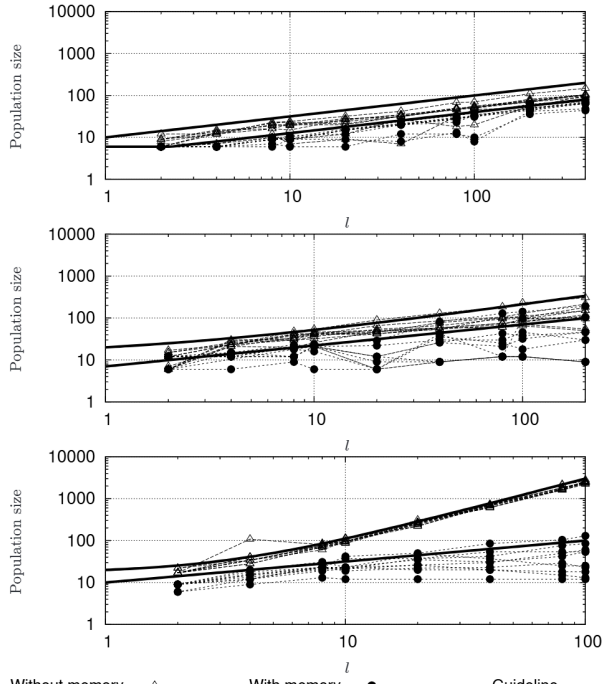


Figure 2: Optimal population size and guideline for AMaLGaM for each benchmark problem and each distribution variant (top: naive; center: Bayesian; bottom: full) with and without the use of memory. With a memory, a smaller guideline is obtained.

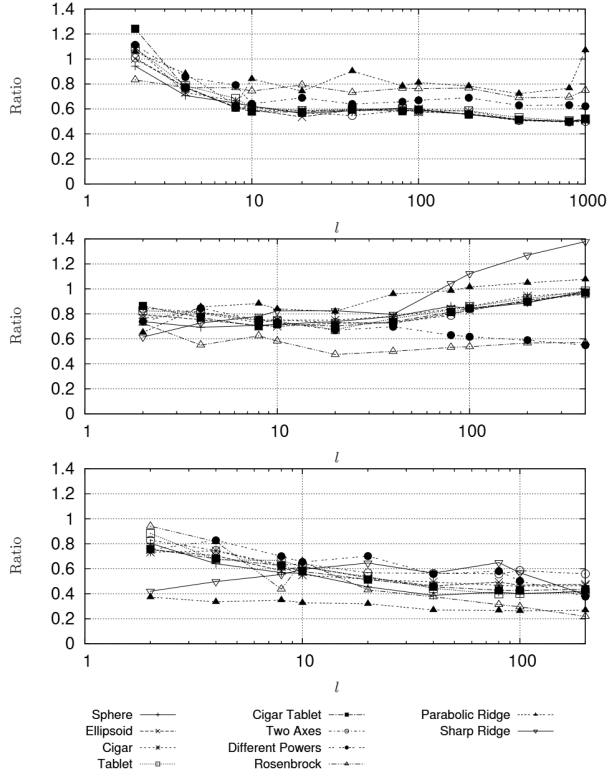


Figure 3: Ratio of evaluations for iAMaLGaM versus AMaLGaM with the population-sizing guideline for each benchmark problem and each distribution variant (top: naive; center: Bayesian; bottom: full).

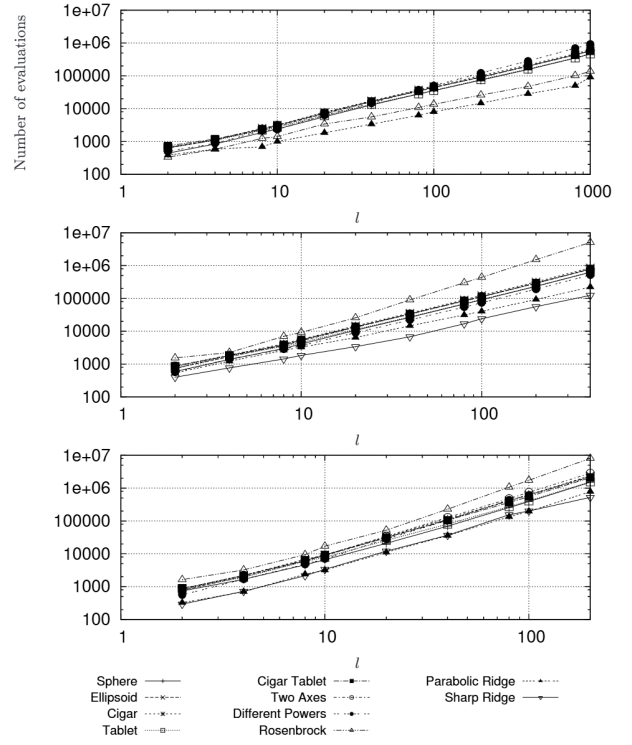


Figure 4: Number of evaluations for iAMaLGaM with the population-sizing guideline for each benchmark problem and each distribution variant (top: naive; center: Bayesian; bottom: full).

3. FASTER SELECTION OF BAYESIAN FACTORIZATIONS

Bayesian factorization selection in EDAs is often done with a greedy algorithm. The most common approach starts with an empty Bayesian network (i.e. the univariate factorization). In each loop of the greedy algorithm, all arcs are considered that do not introduce a cycle in the Bayesian network upon addition. The arc that contributes the most according to a certain metric (e.g. the BIC metric), is added. Assuming that any variable may have at most κ parents, the computational complexity is κl^3 . For each variable, κ times a parent can be added. Because all arcs are considered for each addition, $\mathcal{O}(l^2)$ comparisons are required each time.

For a Gaussian EDA this implies that learning a Bayesian factorization requires the same asymptotic running time as when the unfactorized distribution is used. This is because it takes $\mathcal{O}(l^3)$ time to invert the covariance matrix or compute its Cholesky factorization. The only difference lies in the constants and the fact that a full covariance matrix requires a larger population size to be estimated properly. Bayesian factorization learning for Gaussian EDAs would be more useful if the computational complexity is smaller than $\mathcal{O}(l^3)$.

To this end, we propose to first compute all variances and covariances σ_{ij}^2 , $i, j \in \{0, 1, \dots, l-1\}$. This takes $\mathcal{O}(l^2|\mathcal{S}|)$ time; the same if the full covariance matrix is used. Then, for each variable x_i , sort all $|\sigma_{ij}^2|$, $j \in \{0, 1, \dots, l-1\}$ in descending order. This takes $\mathcal{O}(l^2 \log(l))$ time. Compute a random order of the variables. Then start at the first randomly selected variable and traverse its sorted list. The first variable with a positive contribution to the scoring metric

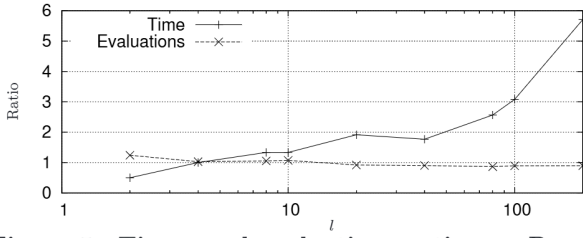


Figure 5: Time- and evaluations-ratios on Rosenbrock for AMaLGaM with traditional greedy algorithm versus new greedy algorithm.

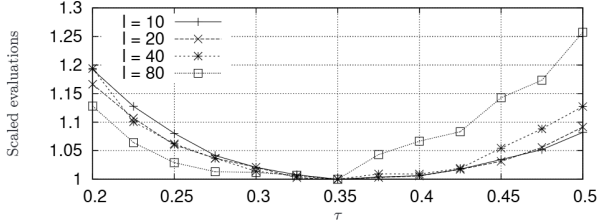


Figure 6: Least number of evaluations, averaged over all benchmark problems using AMaLGaM with full covariance matrix. Results per l scaled to $[1 :]$ by division by least minimum found over all τ .

when added as a parent, is used. The search then continues for more parents, but now for the variable that was added as the parent. If no parent can be found, the overall random order is used to determine the next variable to inspect, continuing in its list where a parent was last found. Each list is thereby traversed exactly once and thus overall $\mathcal{O}(l^2)$ checks for parent addition are done.

The overall runtime is $\mathcal{O}(l^2 \log(l))$. Although this algorithm is faster, it is also more greedy because it performs less comparisons for parent addition. It is therefore expected to give Bayesian factorizations of slightly lesser quality. In Figure 5 both the time-ratio and the evaluations-ratio are plotted for AMaLGaM using the original versus the new greedy algorithm on the Rosenbrock function. The time-ratio benefit is larger than the evaluations-ratio loss, especially for larger l . This benefit will be smaller if evaluations take longer. Throughout this paper (i.e. also in the previous Section) we have used the second greedy algorithm.

4. PARAMETER-FREE GAUSSIAN EDAS

The structure of most parameter-free GEAs is the same (see, e.g. [1, 10, 12]). Upon convergence, a base GEA is restarted with increased resources until a certain overall termination criterion is met. Here we follow the same structure.

4.1 Base single-run EDA

The base single-run EDAs that we use are AMaLGaM, iAMaLGaM and CMA-ES. For AMaLGaM we have changed a few details compared to the literature [3].

First, we changed the truncation selection percentile τ from 0.3 to 0.35. The value of 0.3 has been often used following an older rule of thumb. We ran tests to validate this rule of thumb for AMaLGaM. For each of the problems in Section 2.2.3 the minimally obtainable number of evaluations was determined for a certain value of τ . The results for various values of l and using the full covariance matrix in AMaLGaM are presented in Figure 6. From these results, the choice of $\tau = 0.35$ was derived.

Second, instead of keeping all selected solutions, we propose to keep only the single best solution. This reduces the probability of getting stuck if the selected solutions are spread over multiple local minima. In that case it is hard for a single Gaussian to focus on a single local minimum and thus to obtain further improvements. This has repercussions however on the percentile α of solutions to which to apply AMS in AMaLGaM. In Section 2.2.2 the basic operation of AMS was already recalled. The idea behind AMS is that the solutions changed by AMS are further down the slope. Selecting those solutions as well as solutions not changed by AMS aligns the distribution estimate better with the direction of descent (for illustrations, see [3]). Because we propose now to only keep the best solution, $n - 1$ new solutions are generated to refill the population. To proportion the selected solutions perfectly between unaltered and altered solutions, one obtains $\alpha(n - 1) = \frac{1}{2}\tau n$ and thus $\alpha = \frac{1}{2}\tau \frac{n}{n-1}$. Previously, $\alpha = \frac{\tau}{2-2\tau}$ was used.

Third, a distribution multiplier $c^{\text{Multiplier}}$ is used in AMaLGaM to enlarge the covariance matrix beyond its maximum-likelihood estimate and prevent premature convergence on slopes. If improvements are found more than one standard deviation away from the mean, indicating the search is on a slope, $c^{\text{Multiplier}}$ is enlarged by multiplication with a small number $\eta^{\text{INC}} > 1$. $c^{\text{Multiplier}}$ is decreased by multiplication with $\eta^{\text{DEC}} = 1/\eta^{\text{INC}}$ in case no improvements are found. $c^{\text{Multiplier}}$ was previously not allowed to become smaller than 1. To detect termination, we now *do* allow this. Specifically, we allow $c^{\text{Multiplier}} < 1$ if no improvement has been found in more than a predefined number of subsequent generations. When this happens, the mean of the distribution is set to the best solution. This allows the EDA to focus on a single peak: the one to which the currently best solution belongs. If an improvement is found during this phase, $c^{\text{Multiplier}}$ is reset to 1. Termination is enforced if $c^{\text{Multiplier}} < 10^{-10}$. From previous work, $\eta^{\text{DEC}} = 0.9$ has been found to give good results. The number of subsequent generations before $c^{\text{Multiplier}} < 1$ is allowed, is set to $25 + l$. This number varies with l since it takes more generations to converge if l increases.

4.2 Restarting Scheme

Most parameter-free GEAs employ an exponential population-sizing scheme (see, e.g. [1, 10, 12]). If the population size increases, EDAs with the Gaussian distribution become more robust on slopes induced with “noise”. Typical examples of such problems are Ackley’s function and Griewank’s function. In these massively multimodal problems there is an underlying smooth structure. A large enough population blurs out the noise. Enlarging the population size may not always increase the probability of success however. A good example is Michalewicz’ function (for a definition, see Section 4.3.1). It has multiple local optima, but no underlying structure that is easily smoothed out by using a larger population. Instead, it can be described as having long channels along which the minimum value is the same. Gradient information in such a channel therefore does not lead to better local optima at the intersections of the channels. Proper optimization is therefore only possible if the various intersections are found and optimized. Given that the global optimum is located in the sharpest peak, the population size must be increased extremely before initialization favorably places solutions well inside the global optimum at better values than solutions at other local optima.

From the description of the Michalewicz function it should be clear that confining the search to smaller parts of the search space is a better way of increasing the probability of finding the global optimum. Since this is a type of structure that may equally well appear in a real-world problem, we argue that a different restarting scheme may be useful. Therefore, we propose to allow for parallel executions of the EDA. Given m parallel executions of subpopulation size n^{Sub} (i.e. $mn^{\text{Sub}} = n$), n individuals are initialized in the same way as for a single population. Then, clustering is used to split the solutions into m clusters. In the approach taken here, first the solution is found with the minimum value in a randomly chosen dimension. Then, $m - 1$ further solutions are repeatedly chosen. Each new chosen solution maximizes the minimal distance to the already chosen solutions. These m solutions are called the leaders. The remainder of the solutions are allocated iteratively to the cluster whose leader is nearest and whose size is still smaller than n^{Sub} . For each cluster, a run is then performed independently and in parallel. On a single computing core, the generations are synchronized. The run terminates when all runs have terminated or when any run reaches the VTR.

Increasing the number of solutions can now be done in two ways: a single run with a larger population or more parallel runs. Because a single run with a larger population size is likely to be more useful for problems that have a smooth structure superimposed with noise, we propose to alternate the two schemes. When increasing the number of parallel runs, we propose to also increase the subpopulation size slightly so as to increase the robustness of the more localized searches. Let t^{Restart} denote the number of restarts so far (i.e. the first run has $t^{\text{Restart}} = 0$). We then propose:

- t^{Restart} even: $n^{\text{Sub}} = \lfloor t^{\text{Restart}}/2 \rfloor n^{\text{Base}}$, $m = 2^{\lfloor t^{\text{Restart}}/2 \rfloor}$
- t^{Restart} odd: $n^{\text{Sub}} = 2^{1 + \lfloor t^{\text{Restart}}/2 \rfloor} n^{\text{Base}}$, $m = 1$

where n^{Base} is a baseline population size (e.g. for AMaL-GaM it is the guidelines provided in Section 2.2.4).

4.3 Experiments

4.3.1 Experimental setup

We used the following set of three multimodal benchmark problems in dimensionalities $l \in \{5, 10, 30\}$ to be minimized.

Name	Definition	Range	VTR
Griewank	$\frac{1}{4000} \sum_{i=0}^{l-1} (x_i - 100)^2 - \prod_{i=0}^{l-1} \cos\left(\frac{x_i - 100}{\sqrt{i+1}}\right) + 1$	$[-600, 600]$	10^{-10}
Michalewicz	$-\sum_{i=0}^{l-1} \sin(x_i) \sin^{20}\left(\frac{(i+1)x_i^2}{\pi}\right)$	$[0, \pi]$	-4.664266718 ($l=5$) -9.563983748 ($l=10$) -28.75472728 ($l=30$)
Rastrigin	$\sum_{i=0}^{l-1} (x_i^2 - 10 \cos(2\pi x_i) + 10)$	$[-10, 5]$	10^{-10}

The Griewank and Rastrigin functions are problems of the type where a underlying smooth structure is superimposed with noise to create local optima. The noise in the Rastrigin function is relatively more profound. We additionally use a rotated version of the Rastrigin function in which all variables are first rotated at a 45 degree angle before function evaluation takes place. This gives essentially the same function but introduces dependencies between all problem variables. Rastrigin doesn't contain highly elongated structures. Therefore, methods that do not exploit dependencies can still optimize the rotated variant of the problem, but

only if dependencies can be tackled properly (i.e. using the full covariance matrix) the performance should be the same.

No analytical formula is known to describe the optimum of the Michalewicz function. It is however dimension-wise decomposable. In a single dimension i it is relatively straightforward to determine the minimum x_i^* with high precision. In l dimensions, the optimum is then just $(x_0^*, x_1^*, \dots, x_{l-1}^*)$. Because the problem difficulty increases strongly with the dimensionality, we relax the VTR as l increases. Specifically, we use a VTR of 0.999^l times the true optimum. This gives the set of VTRs in the table above.

For each variant of AMaL-GaM and iAMaL-GaM and for CMA-ES we performed 1000 individual runs of the parameter-free scheme. For CMA-ES we additionally used a scheme similar to G-CMA-ES where only the population size is increased exponentially. The base population size used for CMA-ES is $4 + \lfloor 3 \log(l) \rfloor$, conforming to the literature [9]. In each run, $l \cdot 10^6$ evaluations are allowed.

4.3.2 Results

The averaged results are presented in Tables 3 and 4. The shaded entries indicate that at least in one run the EDA failed to locate the optimum. For iAMaL-GaM^{Naive} this amounts to 1 run for R30 and 6 runs for RR30. For CMA-ES: 2 runs for M30. The only substantial failure was observed for CMA-ES^{Pop} for which only the population size is increased exponentially: 376 for M5, 250 for M10. Entries marked with † in Table 3 were not found to statistically significantly differ from the best result; a Mann-Whitney U test was used at a significance level of 5%.

The results appear to indicate a contradictory conclusion compared to the Section on memories in that the non-incremental versions require less evaluations and less time than the incremental versions. This is most likely because of the larger base-population size that is used in the non-incremental versions. The incremental version of the naive AMaL-GaM is sometimes even unable to find the optimum because it has a very small base population size. Hence, if memory-resources are not very important, a larger base-population size helps in optimizing multimodal problems.

The effectivity of the exponentially-increasing population size scheme on smooth functions that are superimposed with a certain amount of noise to generate local optima is especially clear on the Rastrigin function. On Griewank it also outperforms or performs equally well compared to the space-division restart scheme, but the difference is less clear. The effectivity of the space-division restart scheme is evident when observing results on the Michalewicz function. There are substantial failures for $l = 5$ and $l = 10$ dimensions if only the population size is exponentially increased. For $l = 30$ the situation is somewhat reversed. There are only two failures however for the division scheme. Moreover, all results for $l = 30$ clearly show that a larger population size is favored. This may indicate a reduced effectivity of the division scheme in higher dimensionalities, but it may also be the effect of using a too large relaxation on the optimum for the VTR. Overall we conclude that the space-division scheme is more generally applicable because in this test it never leads to a substantial number of unsuccessful runs.

The results on the rotated version of the Rastrigin function validate that only CMA-ES and the full version of AMaL-GaM are rotation-invariant. The results are similar to those obtained without rotation. Overall, there is little to

EDA	G5	G10	G30	M5	M10	M30	R5	R10	R30	RR5	RR10	RR30
AM-N	34.7	10.2	15.1	41.0	202.3	587.0	17.9	98.8	765.9	29.7	185.1	1554.0
iAM-N	113.5	19.6	12.1	27.4	422.6	2821.8	49.3	303.3	1609.6	80.0	668.3	3808.2
AM-B	[†] 28.4	8.6	22.8	108.3	281.3	392.2	17.5	84.2	[†] 586.4	25.3	136.5	1180.9
iAM-B	38.7	10.4	17.4	21.5	232.1	699.2	25.2	132.1	896.7	34.3	209.9	1670.0
AM-F	25.1	15.0	89.1	107.5	481.4	222.7	24.1	108.7	699.9	25.0	110.1	711.7
iAM-F	37.7	14.5	41.4	33.9	335.8	749.5	36.0	230.7	2646.9	35.6	233.2	2702.1
CMA	84.4	22.3	10.9	24.7	507.2	4252.4	39.4	192.6	1200.3	41.0	190.6	1188.2
CMA ^{Pop}	33.0	21.1	[†] 11.1	420.3	99.6	652.2	19.9	77.0	553.7	21.2	79.7	552.0

Table 3: Average number of evaluations ($\cdot 10^3$) of successful runs. Best results are bold. Shaded entries have failed runs. Entries marked with [†] are not statistically significantly different from the best.

EDA	G5	G10	G30	M5	M10	M30	R5	R10	R30	RR5	RR10	RR30
AM-N	0.15	0.12	0.25	0.23	1.26	9.20	0.12	0.45	7.20	0.14	0.90	23.48
iAM-N	0.37	0.14	0.20	0.17	2.57	44.49	0.18	1.22	15.30	0.30	3.08	56.65
AM-B	0.14	0.13	0.58	0.47	1.94	9.69	0.12	0.48	9.10	0.14	0.82	24.20
iAM-B	0.18	0.14	0.58	0.16	1.74	18.59	0.14	0.76	15.42	0.19	1.28	36.70
AM-F	0.14	0.14	1.69	0.44	3.13	5.27	0.14	0.54	11.22	0.13	0.63	15.31
iAM-F	0.16	0.14	0.81	0.19	2.25	17.55	0.14	1.07	42.74	0.16	1.26	58.39
CMA	0.50	0.31	0.84	0.30	5.14	167.37	0.32	1.43	34.96	0.29	1.50	41.96
CMA ^{Pop}	0.15	0.25	0.83	2.29	1.41	27.06	0.14	0.48	12.15	0.14	0.56	15.71

Table 4: Average number of seconds of successful runs. Best results are in bold. Shaded entries had unsuccessful runs.

choose between them. Because of the influence of the difference in base-population size, it is even hard to say whether allowing less dependencies gives better results if we only look at the number of evaluations. Only in the time table an advantage becomes clear in that the Naive and Bayesian versions are faster. Again, it should be noted that in practice the complexity of the evaluation function may play a crucial role in such a comparison based on time.

5. SUMMARY AND CONCLUSIONS

We have discussed various ways to improve the efficiency of EDAs. A memory can be used to reduce population size requirements. The approach taken here is empirical in nature, but also quite general and may therefore well be used in other EDAs. Greedy Bayesian factorization selection was sped up for Gaussian EDAs and a new scheme for real-valued parameter-free optimization was proposed. Overall, the results are encouraging and may bring EDAs closer to (easy) application. In future work we intend to do just that: validate the efficiency and effectiveness of EDAs for practical optimization problems; typically dynamic and/or multi-objective problems. For these cases, a small population size and a small generational running time is vital.

A final point we would like to make is that when thinking of real-world applications (of EDAs or GEAs in general), it is of importance to realize that it is becoming common even for home computers to have multiple computing cores. A focus on utilizing this parallel processing power is thus relevant. The space-division scheme in this paper can already be parallelized in a straightforward manner, but a next important step toward making EDAs more practically applicable in everyday computing is to parallelize more aspects as well (e.g. model building and function evaluation).

6. REFERENCES

- [1] A. Auger and N. Hansen. A restart CMA evolution strategy with increasing population size. In D. Corne et al., editors, *Proceedings of the IEEE Congress on Evolutionary Computation — CEC-2005*, pages 1769–1776, Piscataway, New Jersey, 2005. IEEE Press.
- [2] S. Baluja and R. Caruana. Removing the genetics from the standard genetic algorithm. In A. Frieditis and S. Russell, editors, *Proceedings of the Twelfth International Conference on Machine Learning*, pages 38–46, Madison, Wisconsin, 1995. Morgan Kaufman.
- [3] P. A. N. Bosman, J. Grahl, and D. Thierens. Enhancing the performance of maximum-likelihood Gaussian EDAs using anticipated mean shift. In G. Rudolph et al., editors, *Parallel Problem Solving from Nature — PPSN X*, pages 133–134, Berlin, 2008. Springer-Verlag.
- [4] P. A. N. Bosman and D. Thierens. The balance between proximity and diversity in multiobjective evolutionary algorithms. *IEEE Transactions on Evolutionary Computation*, 7(2):174–188, 2003.
- [5] J. Branke. *Evolutionary Optimization in Dynamic Environments*. Kluwer, Norwell, Massachusetts, 2001.
- [6] J. Branke, T. Kauffler, C. Schmidt, and H. Schmeck. A multi-population approach to dynamic optimization problems. In I.C. Parmee, editor, *Adaptive Computing in Design and Manufacture — ACDM 2000*, pages 299–308, Berlin, 1999. Springer-Verlag.
- [7] K. Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons, Chichester, 2001.
- [8] T. S. P. C. Duque, D. E. Goldberg, and K. Sastry. Improving the efficiency of the extended compact genetic algorithm. In C. Ryan et al., editors, *Proceedings of the Genetic and Evolutionary Comp. Conf. — GECCO-2008*, pages 467–468, New York, New York, 2008. ACM Press.
- [9] N. Hansen, S. D. Müller, and P. Koumoutsakos. Reducing the time complexity of the derandomized evolution strategy with covariance matrix adaptation (CMA-ES). *Evolutionary Computation*, 11(1):1–18, 2003.
- [10] G. Harik and F. Lobo. A parameter-less GA. In W. Banzhaf et al., editors, *Proc. of the Genetic and Evol. Comp. Conf. — GECCO-1999*, pages 258–265, San Francisco, California, 1999. Morgan Kaufmann.
- [11] J. A. Lozano, P. Larrañaga, I. Inza, and E. Bengoetxea. *Towards a New Evolutionary Computation. Advances in Estimation of Distribution Algorithms*. Springer-Verlag, Berlin, 2006.
- [12] M. Pelikan and T.-K. Lin. Parameter-less hierarchical BOA. In K. Deb et al., editors, *Proceedings of the Genetic and Evolutionary Computation Conference — GECCO-2004*, pages 24–35, Berlin, 2004. Springer-Verlag.
- [13] M. Pelikan, K. Sastry, and E. Cantú-Paz. *Scalable Optimization via Probabilistic Modeling: From Algorithms to Applications*. Springer-Verlag, Berlin, 2006.
- [14] M. Pelikan, K. Sastry, and D. E. Goldberg. iBOA: the incremental Bayesian optimization algorithm. In C. Ryan et al., editors, *Proceedings of the Genetic and Evolutionary Computation Conference — GECCO-2008*, pages 455–462, New York, New York, 2008. ACM Press.
- [15] R. Ros and N. Hansen. A simple modification in CMA-ES achieving linear time and space complexity. In G. Rudolph et al., editors, *Parallel Problem Solving from Nature — PPSN X*, pages 296–305, Berlin, 2008. Springer-Verlag.
- [16] S. Rudlof and M. Köppen. Stochastic hill climbing with learning by vectors of normal distributions. In T. Furuhashi, editor, *Proceedings of the First Online Workshop on Soft Computing — WSC1*, pages 60–70, Nagoya, 1996. Nagoya University.
- [17] G. Schwarz. Estimating the dimension of a model. *Annals of Statistics*, 6(2):461–464, 1978.
- [18] M. Sebag and A. Ducoulombier. Extending population-based incremental learning to continuous search spaces. In A. E. Eiben et al., editors, *Parallel Problem Solving from Nature — PPSN V*, pages 418–427, Berlin, 1998. Springer-Verlag.
- [19] V. Vapnik. *The Nature of Statistical Learning Theory*. Springer-Verlag, Berlin, 1995.